

Blueseatra - Documentation (EN)

AI-Assisted B2B Quoting SaaS Platform

FastAPI • React • Supabase / PostgreSQL

*Multi-tenant application (FastAPI + React + **Supabase/PostgreSQL**) that turns client requests ("work orders", "quote requests" as PDF/image/text) into **professional Pro Forma quotes**, using AI extraction, a flexible price catalog and an advanced quote editor with PDF generation matching a precise business template.*

Table of Contents

- [1 Overview](#)
- [2 Features](#)
- [3 Architecture & tech stack](#)
- [4 Project structure](#)

- [5 Data model](#)
- [6 Environment variables](#)
- [7 Local installation \(step by step\)](#)
- [8 Supabase database / migration](#)
- [9 Security](#)
- [10 User guide](#)
- [11 API reference](#)
- [12 Frontend \(routes & pages\)](#)
- [13 Scripts & maintenance](#)
- [14 Deployment](#)
- [15 Changelog \(steps completed\)](#)
- [16 Troubleshooting / FAQ](#)

1. Overview

Blueseatra is a **multi-company (multi-tenant)** platform for SMBs in construction / all-trades / maintenance. Each company ("tenant") has its own isolated workspace: users, price catalog, requests and quotes.

Main business flow:

```
Client request (PDF/Image/Text)
  | (AI extraction)
  v
Structured data (client, site, object, lines...)
  | (catalog matching)
  v
Draft quote --> Quote editor --> Validated quote --> Pro Forma PDF
```

2. Features

- **JWT authentication** + multi-tenant (roles: **owner**, **admin**, **operator**, **viewer**, **billing_admin**).
- **AI extraction** of documents (PDF / image / text) into structured data (via Emergent LLM).
- **"Open" catalog**: CSV import of **any column format** (auto-detection + adjustable manual mapping), with every column preserved as dynamic attributes.
- **Catalog management**: versions, activate/deactivate, delete, editable client code.
- **Professional quote editor**: typed lines (Labor, Material, Note, Page break), per-line VAT, margin hidden on the PDF, numeric quantities, catalog item picker.
- **Pro Forma PDF generation** matching a precise business template (ReportLab).
- **Company profile** (legal name, registration number, IBAN, mentions...) injected into the PDF.
- **Audit log** of all sensitive actions.
- **Internationalization** FR / EN (react-i18next).

3. Architecture & tech stack

Layer	Technology
Frontend	React 18, React Router, Tailwind CSS, Shadcn/UI (Radix), lucide-react, Sonner, axios, react-i18next
Backend	FastAPI (Python 3.11), Uvicorn, Pydantic v2
Database	Supabase PostgreSQL 17 (via SQLAlchemy 2 async + asyncpg)
AI / LLM	emergentintegrations (Emergent universal key: OpenAI / Anthropic / Google)
PDF	ReportLab
Auth	JWT (python-jose) + bcrypt hashing (passlib)
Encryption	cryptography (Fernet) for secrets at rest
Process mgr	supervisor (backend + frontend)

Runtime layout

- **Frontend (port 3000)** -> calls the backend via **REACT_APP_BACKEND_URL** with the **/api** prefix.
- **Backend (port 8001)** bound to **0.0.0.0:8001**. All routes are prefixed with **/api**.
- **Kubernetes ingress**: routes **/api/*** -> backend (8001), everything else -> frontend (3000).
- **Backend -> PostgreSQL**: direct connection (asyncpg) via **DATABASE_URL** (Supabase Transaction Pooler).

*Important note: the app does **NOT** use Supabase's PostgREST REST API. It connects directly to PostgreSQL with the owner role. An **adapter** (**pg_adapter.py**) exposes a "motor/MongoDB"-compatible API on top of SQLAlchemy, which made it possible to migrate the database **without rewriting** the business logic in **server.py**.*

4. Project structure

```
/app
|-- backend/
|   |-- server.py           # FastAPI app: routes, auth, business logic
|   |-- ai_service.py       # AI extraction (Emergent LLM)
|   |-- matching.py         # Line <-> catalog matching + totals computation
|   |-- pdf_service.py      # Pro Forma PDF generation (ReportLab)
|   |-- database.py         # Async SQLAlchemy engine (reads DATABASE_URL)
|   |-- models_sql.py       # 14 ORM models (JSONB for dynamic data)
|   |-- pg_adapter.py       # motor-compatible adapter => SQLAlchemy/PostgreSQL
|   |-- migrate_mongo_to_supabase.py # MongoDB => Supabase migration (one-shot, idempotent)
|   |-- enable_rls.py       # Enables RLS + deny-all policy on all tables
|   |-- requirements.txt    # Python dependencies (managed via pip freeze)
|   |-- SUPABASE_MIGRATION.md # Dedicated migration guide
|   |-- .env               # Secrets & config (NOT versioned)
|-- frontend/
|   |-- src/
|   |   |-- App.js         # Route definitions
|   |   |-- i18n.js        # FR / EN translations
|   |   |-- lib/api.js     # axios client (base = REACT_APP_BACKEND_URL + /api)
|   |   |-- pages/        # Landing, Auth, Dashboard, Requests, Catalogs, QuoteEditor...
|   |   |-- components/ui/ # Shadcn/UI components
|   |-- package.json       # JS dependencies (managed via yarn)
|   |-- .env              # REACT_APP_BACKEND_URL (DO NOT modify)
|-- design_guidelines.md
-- README.md
```

5. Data model

14 PostgreSQL tables (primary keys as **UUID** strings). Dynamic/nested fields are stored as **JSONB**.

Table	Purpose	JSONB fields
users	User accounts (unique email, bcrypt password_hash)	—
tenants	Companies / workspaces	—
tenant_users	User <-> tenant membership + role	—
catalogs	Catalogs (name, client_code , active_version_id)	—
catalog_versions	Catalog versions (status, mapping...)	columns , mapping
pricing_items	Catalog items (label, price, VAT, margin...)	suppliers , attributes
requests	Client requests + AI extraction	extracted
quotes	Quotes (client, site, totals...)	lines , meta , pricing_snapshot
quote_versions	Quote version history	snapshot
import_jobs	CSV import logs	—
import_errors	Per-row import errors	raw
audit_logs	Audit log	meta
settings_integrations	Per-tenant AI/n8n settings (AI key encrypted)	—
company_profiles	Company profile for the PDF	—

*Multi-tenant isolation: every query filters by **tenant_id**, and every mutation is preceded by an ownership check (an id from another tenant returns 404).*

6. Environment variables

backend/.env

Variable	Description
DATABASE_URL	Supabase Transaction Pooler URI (port 6543). Required .
SUPABASE_URL	Supabase project URL (<a href="https://<ref>.supabase.co">https://<ref>.supabase.co)
SUPABASE_ANON_KEY	Public anon key (reserved for future Auth/Storage use)
SUPABASE_SERVICE_ROLE_KEY	service_role key (highly sensitive , server only)
JWT_SECRET	JWT signing secret — must be strong (>= 32 chars, non-default)
APP_ENCRYPTION_KEY	Fernet key (base64) to encrypt secrets at rest
EMERGENT_LLM_KEY	Emergent universal key (OpenAI/Anthropic/Google)
MAX_UPLOAD_SIZE	(optional) Max upload size in bytes (default 15 MB)
CORS_ORIGINS	(optional) Allowed origins, comma-separated (e.g. https://blueseatra.com)
MONGO_URL, DB_NAME	Kept for the migration script (MongoDB source)

*Never commit .env. The backend **refuses to start** if **JWT_SECRET** is weak/default.*

frontend/.env

Variable	Description
REACT_APP_BACKEND_URL	Public backend URL. DO NOT modify (platform-managed).

Generate strong secrets

```
# JWT_SECRET
python -c "import secrets; print(secrets.token_urlsafe(48))"
# APP_ENCRYPTION_KEY (Fernet)
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

7. Local installation (step by step)

Prerequisites

- Python **3.11**
- Node.js **20** + **Yarn** (do not use **npm**)
- A **Supabase** project (free) OR an accessible PostgreSQL

1) Get the code

```
git clone <your-repo> blueseatra && cd blueseatra
```

2) Backend

```
cd backend
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install emergentintegrations --extra-index-url https://d33sy5i8bnduwe.cloudfront.net/simple/
```

Create **backend/.env** (see section 6) with at least

DATABASE_URL, **JWT_SECRET**, **APP_ENCRYPTION_KEY**, **EMERGENT_LLM_KEY**.

3) Initialize the database

```
# Create schema + enable RLS
python migrate_mongo_to_supabase.py --create-tables
# (Optional) migrate existing data from MongoDB
python migrate_mongo_to_supabase.py
```

4) Run the backend

```
uvicorn server:app --host 0.0.0.0 --port 8001 --reload
```

5) Frontend

```
cd ../frontend
yarn install
# frontend/.env : REACT_APP_BACKEND_URL=http://localhost:8001
yarn start
```

App available at **http://localhost:3000**.

On this platform, **supervisor** manages both services: **sudo supervisorctl restart backend frontend**.
Hot-reload is on (no restart needed for simple code changes, only for **.env**/dependencies).

8. Supabase database / migration

Connection (critical)

You MUST use the **Transaction Pooler** URI (port **6543**), found in

Supabase -> **Connect** -> *Transaction Pooler* tab:

```
postgresql://postgres.<ref>:<PASSWORD>@aws-0-<region>.pooler.supabase.com:6543/postgres
```

The "Direct Connection" URI ([db.<ref>.supabase.co:5432](#)) does **not** work in this environment (IPv4 resolution unavailable). The `asyncpg` engine is configured with `statement_cache_size=0` (required with the pooler in *transaction* mode).

MongoDB -> Supabase migration steps

```
cd backend
python migrate_mongo_to_supabase.py --create-tables # 1) schema + RLS
python migrate_mongo_to_supabase.py                # 2) copy data (idempotent)
```

The script:

- deletes **nothing** in MongoDB (safety);
- is **idempotent** (upsert by primary key — safe to re-run);
- automatically projects each Mongo document onto the model columns.

More details in [backend/SUPABASE_MIGRATION.md](#).

9. Security

Measures in place (audited):

Area	Measure
JWT	Strong random secret; refuses to start if secret is weak/default/<32 chars.
Passwords	bcrypt hashing (passlib); never stored in clear.
Roles	Role is re-checked in DB (tenant_users), never read from the token.
Multi-tenant isolation	Systematic tenant_id filtering + ownership check (404 otherwise).
Secrets at rest	Tenant AI key encrypted (Fernet) , <code>enc::</code> prefix; never returned to client.
CORS	allow_credentials enabled only with explicit origins (never with wildcard).
Upload	15 MB limit (HTTP 413) on documents and CSV.
PostgreSQL / Supabase	RLS enabled + deny_all policy + anon/authenticated privileges revoked on all 14 tables. The public REST API can read/write nothing.

The app uses a **direct PostgreSQL connection** (owner role that **bypasses RLS**), so RLS has no impact on functionality while it closes the door to the public API.

Remaining recommendations (non-blocking)

- Rate-limiting on `/api/auth/login` (anti brute-force).
- Short single-use PDF token (download currently uses `?token=`).
- Stronger password policy (8+ chars, complexity).
- In prod: set `CORS_ORIGINS=https://blueseatra.com`.

10. User guide

1 **Create an account** (/signup) -> a company workspace (tenant) is created automatically, with a demo catalog.

1 **Import your catalog** (Catalogs -> *Import CSV*):

2 Step 1: drop the CSV file (any column format, , or ; separator).

3 Step 2: check the **preview** and the **auto-detected mapping** (adjustable). Only the *Label/Designation* field is required.

1 Step 3: confirm -> the catalog becomes active. All original columns are preserved.

2 **Create a request** (Requests): upload a "work order" PDF/image or paste text -> the AI extracts structured data.

1 **Generate a quote**: from a request, create a **draft**; lines are pre-matched with the catalog.

2 **Edit the quote** (Editor): add/remove lines (Labor, Material, Note, Page break), set quantities/VAT/margin, pick catalog items.

1 **Fill the company profile** (Settings): legal name, registration number, IBAN, mentions...

2 **Validate** the quote, then **download the Pro Forma PDF**.

3 **Manage the team** (Members): invite users, set roles.

4 **Track activity** (Audit log).

11. API reference

All routes are prefixed with /api and (except signup/login) protected by

Authorization: Bearer <token>.

Auth & members

Method	Route	Description
POST	/api/auth/signup	Create account + tenant
POST	/api/auth/login	Log in (returns a token)
GET	/api/auth/me	Profile + accessible tenants
POST	/api/auth/switch-tenant/{tenant_id}	Switch active tenant
GET / POST	/api/members	List / invite members
PATCH	/api/members/{user_id}	Update a role

Settings & profile

Method	Route	Description
GET / PUT	/api/settings/integrations	AI/n8n settings (AI key encrypted, never returned)
GET / PUT	/api/company-profile	Company profile (PDF)

Requests

Method	Route	Description
POST	/api/requests	Create a request (file/text)
GET	/api/requests	List
GET	/api/requests/{id}	Detail
POST	/api/requests/{id}/process	(Re)run AI extraction

Catalogs

Method	Route	Description
GET	/api/catalogs	List (with versions)
GET	/api/catalogs/{id}/items	Items + dynamic columns + mapping
GET	/api/catalog-template.csv	Downloadable CSV template
POST	/api/catalogs/import/preview	Preview + auto-detected mapping
POST	/api/catalogs/import	Import (adjustable mapping)
GET	/api/import-jobs/{job_id}/errors	Errors of an import
POST	/api/catalogs/{id}/activate/{version_id}	Activate a version
PATCH	/api/catalogs/{id}	Update (e.g. <code>client_code</code>)
POST	/api/catalogs/{id}/deactivate	Deactivate
DELETE	/api/catalogs/{id}	Delete (cascade)
GET	/api/catalog/active	Active catalog (for the quote picker)

Quotes

Method	Route	Description
POST	/api/quotes/draft	Create a draft from a request
GET	/api/quotes	List
GET	/api/quotes/{id}	Detail
PATCH	/api/quotes/{id}	Edit (lines, VAT, margin...)
POST	/api/quotes/{id}/validate	Validate (freezes a version)
POST	/api/quotes/{id}/send	Mark as sent
GET	/api/quotes/{id}/pdf	Download the Pro Forma PDF

Misc

Method	Route	Description
GET	/api/dashboard	Metrics
GET	/api/audit	Audit log
GET	/api/	Healthcheck

cURL example

```
# Login
curl -X POST "$BACKEND/api/auth/login" -H "Content-Type: application/json" \
  -d '{"email":"demo@blueseatra.com","password":"secret123"}'
# List catalogs
curl "$BACKEND/api/catalogs" -H "Authorization: Bearer <TOKEN>"
```

12. Frontend (routes & pages)

Route	Page	Access
/	Landing	Public
/login, /signup	Authentication	Public
/app	Dashboard	Protected
/app/requests, /app/requests/:id	Requests / detail	Protected
/app/catalogs, /app/catalogs/import	Catalogs / import	Protected
/app/quotes, /app/quotes/:id	Quotes / editor	Protected
/app/members	Members	Protected
/app/audit	Audit log	Protected
/app/settings	Settings / profile	Protected
/app/billing	Billing	Protected

- Internationalization: `frontend/src/i18n.js` (FR/EN).
- API client: `frontend/src/lib/api.js` (base `REACT_APP_BACKEND_URL` + `/api`, token injection).
- Build check: `npx esbuild src/ --loader:.js=jsx --bundle --outfile=/dev/null` (never `npm`).

13. Scripts & maintenance

Script	Usage
<code>python migrate_mongo_to_supabase.py --create-tables</code>	Create schema + enable RLS
<code>python migrate_mongo_to_supabase.py</code>	Migrate data MongoDB -> Supabase (idempotent)
<code>python enable_rls.py</code>	(Re)apply RLS + <code>deny_all</code> policy + revoke <code>anon/authenticated</code>
<code>pip freeze > requirements.txt</code>	Update backend dependencies (after <code>pip install</code>)
<code>yarn add <pkg></code>	Add a frontend dependency
<code>sudo supervisorctl status / restart backend frontend</code>	Manage services
<code>tail -n 100 /var/log/supervisor/backend.*.log</code>	Backend logs

14. Deployment

- 1 **Check** deployability (no hardcoded secrets, ports, CORS, build).
- 2 **Deploy** via Emergent's **Deploy** button (Preview -> Deploy -> Deploy Now).
- 3 **Custom domain** (`blueseatra.com`): Dashboard -> Connect -> **Entri** integration

(guided DNS setup).

- 1 In prod: set `CORS_ORIGINS=https://blueseatra.com` and keep `JWT_SECRET` / `APP_ENCRYPTION_KEY`

out of the Git repository.

Platform constraints: backend bound to `0.0.0.0:8001`, `/api/*` routes, services via supervisor, never modify `REACT_APP_BACKEND_URL` or the ingress config.

15. Changelog (steps completed)

Phase 1-2 — MVP & foundation

- JWT auth, multi-tenant, frontend routing, dashboard.
- AI extraction of requests (PDF/image/text) -> structured data.
- Pro Forma PDF generation matching the business template.

Phase 3 — Professional quote editor

- Typed lines (Labor, Material, Note, Page break), per-line VAT, margin hidden on PDF, numeric quantities, A4 adaptation, catalog item picker.
- Hierarchical catalog import (Family -> Item -> Supplier).

Phase 4 — "Open" catalog (dynamic CSV)

- Column auto-detection (FR/EN synonyms) + adjustable mapping in the import wizard.
- Robust parsing (, / ; / tab separator, encodings, 12, 50 decimals).
- All columns preserved in `attributes` (JSONB); dynamic columns display + details.
- **Deactivate** / **Delete** catalog buttons, **editable client code**, UI alignment.
- Fixed display of numeric fields (Qty/Price/Margin) in the editor.

Security audit

- Strong JWT_SECRET + refusal of default secret; hardened CORS; 15 MB upload limit;

Fernet encryption of the AI key; multi-tenant isolation verified.

Migration to Supabase (PostgreSQL)

- Connection via Transaction Pooler (asyncpg, `statement_cache_size=0`).
- 14 SQLAlchemy models (JSONB for dynamic data) + schema created.
- `pg_adapter.py` **adapter** (motor-compatible) -> `server.py` almost unchanged.
- Migration of existing data (idempotent, without deleting MongoDB).
- Tests: backend 60/61, frontend 95%, overall 97% (1 `update_one` bug fixed).

Supabase hardening

- RLS enabled on all 14 tables, `deny_all` policy, `anon/authenticated` privileges revoked

-> Security Advisor "0 error / 0 warning / 0 info".

16. Troubleshooting / FAQ

Backend refuses to start (JWT_SECRET ...)

-> Set a strong JWT_SECRET (>= 32 chars) in `backend/.env`.

DATABASE_URL not set / DB connection errors

-> Use the **Transaction Pooler** URI (port 6543) with the real **password**. The Direct Connection (5432) does not work here.

prepared statement error with the pooler

-> Already handled (`statement_cache_size=0`). If you change the engine, keep this setting in `*transaction*` mode.

HTTP 413 on upload

-> File > 15 MB. Adjust `MAX_UPLOAD_SIZE` if needed.

Supabase Security Advisor reports RLS issues

-> Run `python enable_rls.py`, then "Rerun linter" in the dashboard.

Old tokens no longer work

-> Normal after a `JWT_SECRET` rotation: log in again.

Do not use `npm` — only `yarn`. Never start servers manually: use **supervisor**.

Blueseatra — documentation maintained in `/app/README.md` (FR) and `/app/README.en.md` (EN). For database migration, see also `backend/SUPABASE_MIGRATION.md`.